NSCS204—Computer Organization and Architecture Lab
Department of Computer Science and Engineering
IIT(ISM) Dhanbad

12.10.2026

Practical Digital Systems in Logisim

Project Objective

Design, implement, and verify the project allocated to you as a self-contained Logisim circuit. Model user actions and sensors with input pins/buttons, numerical information with decimal displays, and actuators with labelled LEDs. Your design must integrate control states, stored data, arithmetic where required, and precise timing.

No.	Project	Page
1	Vending machine with credit, change, and cancellation	3
2	Sequential four-digit door lock with attempt lockout	4
3	Microwave controller with pause/resume and door interlock	5
4	Red–green traffic signal with a 63-second countdown	6

Required Submission

1. **Circuit file:** Submit a working `.circ` file with all subcircuits and any memory contents included. It must open and simulate without missing dependencies.
2. **Design report:** Identify name, roll number, lab section, and allocated project. Include the architecture diagram, state diagram or transition table, datapath, signal/storage widths, input-event logic, timer logic, display method, and reset behavior.
3. **Verification records:** Record input stimuli, relevant clock edges, expected results, observed results, and pass/fail status. Address the project-specific checks and the common tests on page 2.
4. **Screenshots:** Include labelled circuit screenshots and observed outputs for representative tests. Provide a screenshot of the Logisim version and record the clock/tick settings used.

Suggested Report Organisation

Use the following order: problem overview and interfaces; architecture and component choices; controller and datapath; timing, reset, and displays; verification records; and a short explanation of the final operating sequence. Ensure the circuit's signal names match the report and make the main circuit easy to use.

Common Implementation and Testing Requirements

1. **Clock:** Use one rising-edge master clock at **64 complete cycles per simulated second**. Set Logisim Evolution's Auto Tick Frequency to **128 Hz**, with High Duration and Low Duration both **1 tick**. Thus $f_{CLK} = 128/(1 + 1) = 64$ Hz and $\Delta = 1/64$ second. Update state, counters, and registers on master-clock edges using appropriate enables; do not use buttons as clocks.

2. **Reset:** RESET is active HIGH and synchronous, with highest priority. Each rising edge with RESET=1 establishes and holds the project's reset state. The first rising edge after release is a startup boundary: retain reset outputs, initialise timers, and accept no button events. Normal processing begins on subsequent edges. In Project 4, startup is $t = 0$ for the first red interval.

3. **Button events:** All button inputs are active HIGH. Detect each sampled $0 \rightarrow 1$ press once; a held button must not repeat its action. Observe that button LOW after reset before accepting its first event. A button held HIGH across reset must be released and pressed again. Update button history even while its command is ignored, preventing deferred actions from busy-state presses.

4. **Stimulus and sampling:** Inputs are noise-free digital signals; physical debounce is not required. Hold each button press and release across at least two rising edges. Keep data stable when its associated event is sampled. Level sensors must span a rising edge. Interpret events using the pre-edge state and inspect outputs after propagation settles. Apply the stated same-edge priorities.

5. **Timing:** Entry to a timed state starts its timer at zero; the entry/start/resume edge consumes no elapsed period. One second ends after 64 subsequent master periods, three seconds after 192, four seconds after 256, and five seconds after 320. Do not shorten a new interval because a global divider was already partway through its cycle. Preserve fractional timing where required. An ignored command does not interrupt timing unless a specified action does so.

6. **Displays and observability:** Use actual seven-segment displays for decimal values, with leading zeros in two-digit fields and decimal points OFF. Binary or BCD storage is permitted; explain conversion/decoding. Provide labelled output pins/probes for named outputs and sufficient internal probes to verify state and timing.

7. **Components:** Built-in gates, flip-flops, registers, counters, multiplexers, decoders, arithmetic blocks, and RAM/ROM may be used. Explain their configuration and include any ROM contents. No external program may implement the controller; scripts may assist verification. Avoid width conflicts and unintended unknown/error values.

8. **Minimum verification:** Test normal operation, held buttons, simultaneous events, timer boundaries, and reset in every operating state. Hold reset across at least three rising edges and check release with a button held HIGH. Verify normal-speed timing; identify the scale of any accelerated runs. Record actual observations.

Timing reference: A simulation tick is not a complete clock cycle.[1] The stated clock convention fixes the meaning of every "master period" and "second" used in the project requirements.

[1] Logisim Clock component reference.

Project 1: Vending Machine

Problem statement. Design a vending machine for one product costing **Rs. 15**. Accept Rs. 5 and Rs. 10 coin requests, accumulate credit, automatically dispense on sufficient payment, and display change. Cancellation must refund undispensed credit. Assume unlimited product stock and sufficient refund/change reserves. Coin inputs are requests to accept coins; ignored inputs do not represent accepted money.

Interface and Reset State

Signal	Bits	Meaning
`COIN5`, `COIN10`, `CANCEL`	1 each	Press-event inputs.
`CREDIT`	5	Unsigned accepted credit: 0, 5, 10, 15, or 20 rupees.
`RETURN_AMOUNT`	4	Change/refund, 0–10 rupees; two decimal digits.
`DISPENSE`, `REFUND`	1 each	Product-delivery and refund indicators.
`BUSY`, `ERROR`	1 each	Transaction-in-progress and conflicting-coin indicators.

Show CREDIT and RETURN_AMOUNT on separate two-digit displays. Reset establishes **COLLECT**, CREDIT=00, RETURN_AMOUNT=00, and all indicators LOW.

Required Operation

1. **COLLECT:** Accept one coin event on an edge and add its denomination. Both coin events on the same edge are rejected together: retain credit and return amount, set ERROR=1, and remain in COLLECT. No coin-event priority is used.
2. **Below price:** If the updated credit is below 15, retain it and remain in COLLECT. An accepted single coin clears ERROR and clears the previous RETURN_AMOUNT.
3. **Purchase:** If the updated credit reaches 15 or 20, enter VEND on that same edge. Use the updated credit, including the coin just accepted. Set RETURN_AMOUNT=credit − 15. Retain CREDIT at 15 or 20 during VEND.
4. **VEND:** Assert DISPENSE=1 and BUSY=1 for exactly 64 subsequent master periods (one second); REFUND=0. Then return to COLLECT and clear CREDIT. Retain RETURN_AMOUNT until another single coin is accepted or reset occurs. One purchase produces one continuous DISPENSE interval, not repeated dispensing.
5. **Cancellation:** In COLLECT, CANCEL takes priority over either or both coin events and clears ERROR. With positive credit, copy that credit to RETURN_AMOUNT, clear CREDIT immediately, and enter RETURN. With zero credit, remain in COLLECT and retain the previous RETURN_AMOUNT; do not start a refund interval.
6. **RETURN:** Assert REFUND=1 and BUSY=1 for one second; DISPENSE=0. Then return to COLLECT, retaining RETURN_AMOUNT. Ignore all coin and CANCEL events in VEND/RETURN, including their expiry edges. RESET overrides every state and terminates activity.

Verification checks: Exact payment (5+5+5), overpayment (10+10), cancellation of partial credit, simultaneous coin events, CANCEL with a coin event, held inputs, and ignored events during VEND or RETURN and on their expiry edges. BUSY is DISPENSE OR REFUND; the two activity indicators must never be HIGH together.